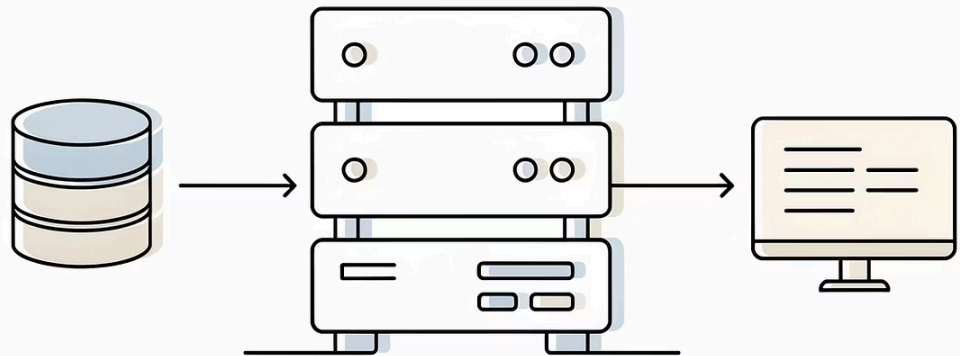




Processamento de Consulta em Bancos de Dados

O processamento de consultas é um dos componentes fundamentais de um Sistema Gerenciador de Banco de Dados (SGBD). Este módulo explora como as consultas são analisadas, otimizadas e executadas para retornar resultados eficientes aos usuários.

Eduardo Ogasawara

eduardo.ogasawara@cefet-rj.br

<https://eic.cefet-rj.br/~eogasawara>

Etapas Básicas no Processamento da Consulta

O processamento de uma consulta em um banco de dados relacional passa por três etapas fundamentais que transformam uma requisição SQL em resultados concretos. Cada etapa desempenha um papel crucial na eficiência e correção do processo.

Análise e Tradução

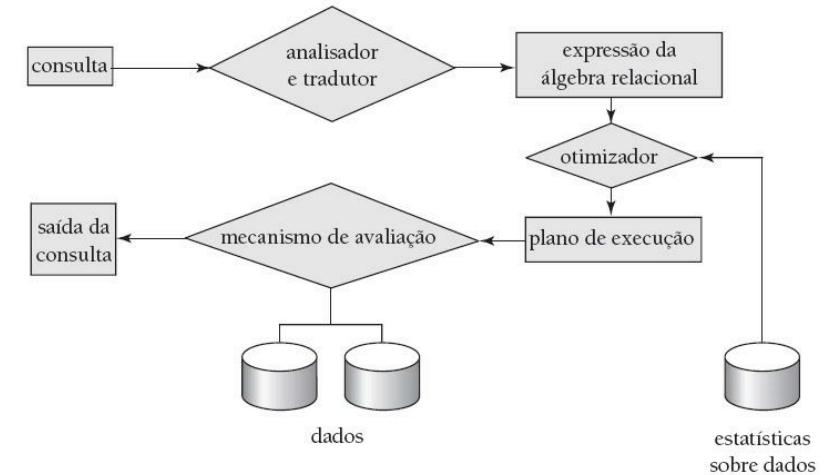
Converte a consulta SQL em uma representação interna e verifica sua validade sintática e semântica

Otimização

Identifica o plano de execução mais eficiente entre múltiplas alternativas equivalentes

Avaliação

Executa o plano escolhido e retorna os resultados ao usuário



Análise, Tradução e Avaliação

Análise e Tradução

Nesta primeira etapa, o sistema realiza uma série de transformações e verificações essenciais:

- A consulta SQL é traduzida para um formato interno do SGBD
- Essa representação interna é convertida em expressões de álgebra relacional
- O analisador sintático verifica se a consulta está gramaticalmente correta
- O analisador semântico valida se as relações e atributos referenciados existem no esquema

Avaliação

A fase final executa o plano selecionado e produz os resultados:

- O mecanismo de execução de consulta recebe o plano de avaliação otimizado
- Executa cada operação do plano na ordem especificada
- Acessa os dados necessários em disco ou na memória
- Retorna as tuplas resultantes à aplicação ou usuário

Etapas Básicas no Processamento da Consulta: Otimização

A otimização de consultas é o coração do processamento eficiente em SGBDs. Esta etapa crítica determina como uma consulta será executada, impactando diretamente no tempo de resposta e no uso de recursos do sistema.

O otimizador analisa múltiplas estratégias de execução, cada uma com diferentes ordenações de operações e métodos de acesso aos dados. Utilizando estatísticas armazenadas no catálogo do banco de dados, ele estima o custo de cada alternativa e seleciona aquela que promete o melhor desempenho.

Árvore de Expressão como Plano de Avaliação

A representação interna de uma consulta como árvore de expressão é fundamental para o processamento eficiente. Esta estrutura hierárquica permite visualizar e manipular as operações da álgebra relacional de forma sistemática.

Estrutura da Árvore

Cada nó interno representa uma operação da álgebra relacional (seleção, projeção, junção), enquanto as folhas representam as relações base do banco de dados

Planos Equivalentes

Múltiplas árvores diferentes podem representar a mesma consulta lógica, produzindo resultados idênticos mas com custos de execução distintos

Escolha Otimizada

O otimizador avalia as alternativas e seleciona a árvore com menor custo estimado, considerando estatísticas e heurísticas de otimização

Processo de Otimização de Consultas

Análise de Alternativas

Entre todos os planos de avaliação equivalentes, o otimizador identifica e compara múltiplas estratégias de execução

Estimativa de Custo

Utiliza informações estatísticas do catálogo (número de tuplas, tamanho de registros, distribuição de valores) para calcular o custo esperado

Seleção do Melhor

Escolhe o plano com menor custo estimado para execução real

Objetivos do Processamento de Consultas

- Desenvolver métricas precisas para medir custos de diferentes estratégias de execução
- Implementar algoritmos eficientes para avaliar operações da álgebra relacional
- Combinar operações individuais em planos de execução completos e otimizados

No módulo de otimização de consultas, o foco está em encontrar o plano de avaliação com menor custo estimado, aplicando técnicas como transformações algébricas, ordenação de operações e seleção de métodos de acesso.

Medidas de Custo da Consulta

O custo de execução de uma consulta é tipicamente medido pelo tempo total necessário para produzir os resultados. Diversos fatores contribuem para este custo, mas o acesso ao disco geralmente domina a métrica.

Componentes do Custo de E/S em Disco

Operações de Busca

Número de buscas × custo médio de busca — O tempo para posicionar o cabeçote de leitura na posição correta

Leitura de Blocos

Número de blocos lidos × custo médio de leitura — A transferência de dados do disco para a memória

Escrita de Blocos

Número de blocos escritos × custo médio de escrita — Operações de escrita são mais custosas que leituras devido a verificações adicionais



Simplificações na Modelagem

Para facilitar a análise, frequentemente utilizamos o **número de transferências de bloco** como medida única de custo.

Podemos ignorar:

- Diferenças entre E/S sequencial e aleatória
- Custos de processamento da CPU
- Custos de comunicação em rede

Influência da Memória nas Medidas de Custo

A quantidade de memória disponível para buffers é um fator crítico que afeta dramaticamente o custo real de execução das consultas. Um buffer maior reduz significativamente a necessidade de acessos ao disco, melhorando o desempenho.

Desafio da Previsão

A quantidade real de memória disponível é difícil de determinar antecipadamente, pois depende de processos concorrentes do sistema operacional e de outras operações do SGBD em execução simultânea.

Estratégia Conservadora

Tipicamente, as estimativas de custo assumem o **pior caso possível**, supondo que apenas a quantidade mínima de memória necessária para executar a operação estará disponível.

Impacto no Desempenho

Ter mais memória disponível pode transformar operações que exigiriam múltiplas passadas pelo disco em operações realizadas completamente em memória, reduzindo drasticamente o tempo de execução.

Dominando as Operações da Álgebra Relacional

Para processar consultas eficientemente, é essencial compreender os algoritmos disponíveis para cada operação fundamental da álgebra relacional. Cada operação pode ser implementada de múltiplas formas, cada uma adequada a diferentes cenários.

Seleção

Filtra tuplas que satisfazem um predicado. Implementações incluem varredura completa de arquivo, uso de índice primário, secundário ou combinação de índices.

Ordenação

Organiza tuplas segundo um ou mais atributos. Algoritmos principais incluem Quick Sort para dados em memória e Merge Sort externo para volumes grandes.

Junção

Combina tuplas de duas relações. Técnicas incluem Loops Aninhados (simples mas custoso), Hash Join (eficiente para grandes volumes) e Merge Join (ótimo para dados ordenados).

Avaliação da Operação de Projeção

A operação de projeção elimina atributos de uma relação, mantendo apenas as colunas especificadas. Embora conceitualmente simples, sua implementação eficiente apresenta desafios importantes.

O principal desafio é a **eliminação de duplicatas**. Quando projetamos sobre atributos que não formam uma chave, tuplas idênticas podem surgir e precisam ser removidas para manter a semântica de conjuntos da álgebra relacional.

Projeção Baseada em Ordenação

Ordena as tuplas projetadas e então remove duplicatas consecutivas em uma única passada. Eficiente quando a ordenação pode ser feita em memória ou com poucas passadas.

O custo da operação depende criticamente do tamanho da relação, da necessidade real de eliminar duplicatas (que pode ser evitada se projetarmos sobre uma chave) e da memória disponível para buffers.

Projeção Baseada em Hashing

Utiliza uma função hash para particionar as tuplas e eliminar duplicatas dentro de cada partição. Adequado quando há memória suficiente para as estruturas de hash.

Varredura Completa: Seleção por Varredura de Arquivo

Varredura Linear Completa

O método mais simples: examina cada bloco do arquivo sequencialmente, testando o predicado de seleção em cada tupla

Custo de E/S

Requer br transferências de bloco, onde br é o número de blocos que a relação ocupa no disco

Quando Usar

Adequado para predicados de baixa seletividade ou quando não há índices disponíveis sobre os atributos do predicado

A varredura de arquivo é o método de acesso mais básico e funciona para qualquer predicado de seleção. Apesar de sua simplicidade, pode ser bastante eficiente quando:

- O predicado tem **baixa seletividade** (seleciona grande porcentagem das tuplas)
- Os dados estão armazenados de forma **sequencial** no disco
- O sistema operacional pode realizar **leitura antecipada** (read-ahead)



Vantagem: Não requer estruturas auxiliares ou índices

Desvantagem: Sempre examina toda a relação, mesmo para predicados altamente seletivos

Acesso Indexado: Seleção via Índices

Índices são estruturas de dados auxiliares que permitem acesso rápido a tuplas específicas sem examinar toda a relação. Existem diversos tipos de índices, cada um otimizado para diferentes padrões de acesso.

Índice Primário

Construído sobre a chave de ordenação do arquivo. Permite busca binária eficiente e acesso sequencial rápido a registros relacionados.

Índice Secundário

Construído sobre atributos não-chave. Pode apontar para múltiplas tuplas e geralmente é mais custoso que índices primários para acessos múltiplos.

Índice Hash

Utiliza função hash para localizar rapidamente tuplas. Excelente para buscas por igualdade, mas não suporta consultas de intervalo.

Escolha entre Varredura e Uso de Índices

A decisão de usar um índice ou varredura sequencial não é trivial. O otimizador deve considerar múltiplos fatores para fazer a escolha correta.

Seletividade do Predicado

Quantas tuplas satisfazem a condição?
Predicados altamente seletivos favorecem índices.

Tipo de Índice Disponível

Índice primário? Secundário? Hash? Cada tipo tem custos diferentes.

Custo Estimado de E/S

Comparação entre o custo de usar o índice versus varrer o arquivo inteiro.



Percepção Importante

Índices **nem sempre são a melhor opção**. Para predicados com baixa seletividade (que retornam muitas tuplas), uma varredura sequencial pode ser mais eficiente do que múltiplos acessos aleatórios via índice.

Seleção com Igualdade: Índice sobre Chave

Quando o predicado de seleção especifica igualdade sobre uma chave primária ou candidata, sabemos que no máximo uma tupla satisfará a condição. Essa garantia de unicidade permite otimizações significativas.

Busca no Índice

O otimizador utiliza o índice (tipicamente uma árvore B+ ou estrutura hash) para localizar rapidamente a entrada correspondente ao valor buscado

Acesso Direto

Uma vez localizada a entrada no índice, um único acesso ao arquivo de dados recupera a tupla desejada

Retorno do Resultado

A tupla é retornada ao usuário. Não são necessárias verificações adicionais devido à restrição de unicidade

Custo de E/S

Para uma árvore B+ de altura h :

- $h + 1$ transferências de bloco no total
- h acessos para navegar no índice
- 1 acesso para recuperar a tupla

Eficiência

Este é um dos cenários mais eficientes de busca em bancos de dados. Com índices bem dimensionados, h é tipicamente 3-4 mesmo para tabelas com milhões de registros.

Seleção com Igualdade: Índice Primário

Quando temos um predicado de igualdade sobre atributos que não formam uma chave, múltiplas tuplas podem satisfazer a condição. Um índice primário (construído sobre a chave de ordenação do arquivo) oferece vantagens especiais neste cenário.

Localização Inicial

O índice primário é usado para encontrar o primeiro registro que satisfaz o predicado de igualdade

Leitura Sequencial

Como o arquivo está ordenado pela chave de indexação, todos os registros que satisfazem a condição estão fisicamente adjacentes

Acesso Eficiente

Registros adicionais são lidos sequencialmente até que a condição não seja mais satisfeita

Custo: $h + b$ acessos a blocos, onde:

- h = altura da árvore do índice
- b = número de blocos contendo registros que satisfazem o predicado

A leitura sequencial é muito mais eficiente que acessos aleatórios, tornando índices primários ideais para predicados com múltiplos resultados.



Vantagem chave: Localidade física dos dados relacionados minimiza movimentos do cabeçote do disco

Seleção com Igualdade: Índice Secundário

Índices secundários são construídos sobre atributos que não são a chave de ordenação do arquivo. Diferentemente de índices primários, os registros correspondentes não estão fisicamente agrupados.

Busca no Índice

Navegação na estrutura do índice secundário (h acessos)

Lista de Ponteiros

Recuperação da lista com n ponteiros para registros

Acessos Aleatórios

Até n acessos ao disco para recuperar cada registro individualmente

Custo: Pior Caso

$h + n$ transferências de bloco

Onde n é o número de registros que satisfazem o predicado. Cada registro pode estar em um bloco diferente.

Custo: Caso Otimista

Se alguns registros compartilham blocos, o custo pode ser menor. Porém, essa otimização é difícil de prever.

 **Atenção:** Para predicados com baixa seletividade, um índice secundário pode ser **menos eficiente** que uma varredura completa devido aos acessos aleatórios.

Seleção com Predicado de Comparação

Predicados de comparação (maior que, menor que, entre outros) requerem estratégias diferentes dos predicados de igualdade. A eficiência do acesso depende fortemente da disponibilidade e tipo de índice.

Com Índice Ordenado

Árvores B+ permitem navegação eficiente. Para $A \geq v$, localiza-se v e percorre-se para frente; para $A \leq v$, percorre-se desde o início até v

Sem Índice Adequado

Varredura completa do arquivo é necessária, examinando cada tupla para verificar se satisfaz a condição de comparação

Custos por Tipo de Acesso

Índice Primário ($\geq$ ou $\leq$)

Custo: $h + b$ blocos, onde b é o número de blocos contendo resultados. Leitura sequencial após localização inicial.

Índice Secundário ($\geq$ ou $\leq$)

Custo: $h + n$ blocos (pior caso), onde n é o número de tuplas. Acessos potencialmente aleatórios a cada registro.

Varredura de Arquivo

Custo: br blocos, onde br é o tamanho total da relação. Exame de todas as tuplas.

Seleção com Predicado Composto: Conjunção

Quando temos múltiplas condições conectadas por AND ($\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n$), todas devem ser simultaneamente verdadeiras. O otimizador pode explorar diferentes estratégias dependendo dos índices disponíveis.

Estratégia 1: Índice Único

Selecione um índice sobre um dos atributos que prometa maior seletividade. Use-o para recuperar tuplas candidatas e então avalie as demais condições em memória.

Estratégia 2: Índice Composto

Se existe um índice composto sobre múltiplos atributos do predicado, use-o para filtrar simultaneamente por várias condições.

Estratégia 3: Interseção de Índices

Use múltiplos índices separados, recupere os conjuntos de identificadores de registro de cada um, compute a interseção e então acesse apenas os registros na interseção.



Escolha da Estratégia

A estratégia 3 (interseção) é vantajosa quando cada condição individual tem baixa seletividade, mas a conjunção é altamente seletiva. A estratégia 1 é preferível quando uma condição já é altamente seletiva.

Seleção com Predicado Composto: Disjunção

Predicados com disjunção ($\theta_1 \vee \theta_2 \vee \dots \vee \theta_n$) são mais desafiadores: basta que uma condição seja verdadeira. Isso limita as oportunidades de otimização em comparação com conjunções.

Com Índices em Todas as Condições

Se há índices disponíveis para todos os atributos mencionados na disjunção:

- Use cada índice para recuperar conjuntos de identificadores
- Compute a **união** dos conjuntos de identificadores
- Acesse os registros correspondentes aos identificadores na união
- Elimine duplicatas se necessário

Custo com Índices Completos

Soma dos custos de uso de cada índice individual, mais o custo de unir os resultados e eliminar duplicatas.

Sem Índices Completos

Se falta índice para qualquer uma das condições na disjunção:

- Necessário realizar **varredura completa** do arquivo
- Avaliar o predicado completo para cada tupla
- Não é possível otimizar parcialmente

Razão: Mesmo que alguns atributos tenham índice, não podemos garantir que capturamos todas as tuplas sem examinar o arquivo inteiro.

Custo sem Índices Completos

br transferências de bloco para varrer toda a relação, independente de quantos índices parciais existam.

Ordenação e Avaliação de Consultas em Bancos de Dados

Uma exploração detalhada das técnicas de ordenação e operações de junção para processamento eficiente de consultas em sistemas de gerenciamento de bancos de dados relacionais.

Estratégias de Ordenação para Relações

Relações que cabem na memória

Quando a relação cabe inteiramente na memória principal, algoritmos como o quicksort podem ser aplicados diretamente, sem necessidade de acessos adicionais ao disco.

- Acesso rápido aos dados
- Sem overhead de I/O
- Ideal para datasets pequenos e médios

Relações que não cabem na memória

Quando a relação excede a memória disponível, utiliza-se o merge-sort externo, que divide os dados em partições e realiza a ordenação em múltiplas passagens pelo disco.

- Operações sequenciais em disco
- Divisão em partições gerenciáveis
- Processamento em múltiplas fases

Sort-Merge: Princípio Geral

Fase 1: Ordenação Inicial

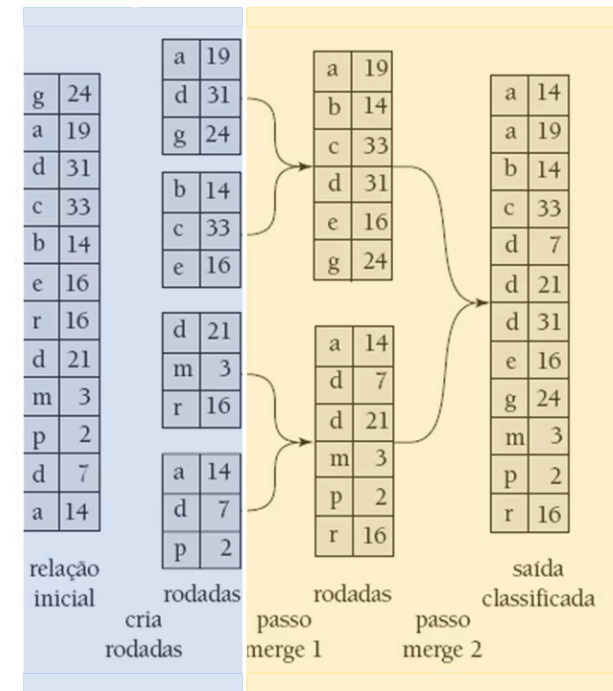
A primeira fase do merge-sort externo envolve a criação de runs ordenados. O algoritmo lê blocos de dados que cabem na memória, ordena-os internamente usando quicksort ou outro algoritmo eficiente, e grava os resultados como runs temporários no disco.

Fase 2: Mesclagem Ordenada

Na segunda fase, os runs ordenados são mesclados iterativamente. Em cada passada, múltiplos runs são combinados em runs maiores, mantendo a ordem global. Este processo continua até que reste apenas um único run ordenado contendo todos os dados.

O diagrama ilustra o fluxo de dados através das duas fases principais: particionamento e ordenação local seguidos por mesclagem progressiva até alcançar a ordenação completa.

1. Fase de ordenação



2. Fase de mesclagem ordenada

Merge-Sort Externo

Ordenar relações **maiores que a memória** em duas fases: criação de runs ordenados e mesclagem progressiva.

Custo Total

$$Custo = b_r \times (2 \times \lceil \log_{M-1}(b_r/M) \rceil + 1)$$

- **b_r** = número de blocos da relação
- **M** = buffers disponíveis em memória

ENTRADA: relação R com b_r blocos; M buffers de memória

PRÉ-CONDIÇÃO: $M \geq 2$

PASSO 1 — Criação de runs ordenados

$i \leftarrow 0$

enquanto R não foi completamente lida:

ler M blocos de R para a memória

ordenar os M blocos internamente

gravar run_i no disco

$i \leftarrow i + 1$

$N_runs \leftarrow i$

PASSO 2 — Mesclagem progressiva

enquanto $N_runs > 1$:

para cada grupo de (M - 1) runs:

abrir (M - 1) buffers de entrada, 1 buffer de saída

mesclar os runs usando heap mínimo sobre os ponteiros

gravar run mesclado no disco

$N_runs \leftarrow \lceil N_runs / (M - 1) \rceil$

SAÍDA: relação R completamente ordenada em disco

PÓS-CONDIÇÃO: número de passadas = $\lceil \log_{\{M-1\}}(b_r / M) \rceil + 1$

Complexidade do Merge-Sort Externo

A análise de complexidade do merge-sort externo considera principalmente o número de acessos a disco, que é o fator dominante no tempo de execução para grandes volumes de dados.

1

Fase de Ordenação

Custo: Cada bloco é lido uma vez e gravado uma vez

Operações I/O: $2 \times (\text{número de blocos})$

Esta fase tem custo linear em relação ao tamanho dos dados de entrada.

2

Fase de Mesclagem

Número de passadas: $\lceil \log_{M-1}(br/M) \rceil$

Custo por passada: $2 \times br$

Onde M é o número de buffers disponíveis e br é o número de blocos da relação.

3

Custo Total

Fórmula: $br \times (2 \times \lceil \log_{M-1}(br/M) \rceil + 1)$

Esta complexidade logarítmica torna o merge-sort externo escalável para grandes volumes de dados.

Avaliação de Operações de Conjunto

Operações Fundamentais

As operações de conjunto são essenciais em consultas relacionais:

- **União ($\cup$):** Combina tuplas de duas relações
- **Interseção ($\cap$):** Retorna tuplas comuns
- **Diferença ($-$):** Remove tuplas de uma relação

Estratégias de Implementação

Duas abordagens principais são utilizadas:

Baseada em Ordenação

Ordena ambas as relações e processa sequencialmente

Baseada em Hashing

Usa funções hash para particionar e comparar eficientemente



Ambas as abordagens requerem pré-processamento: as relações devem estar ordenadas pelo atributo de comparação ou particionadas por uma função de hash consistente.

Operação de Junção

A **junção** (join) é uma das operações mais importantes e custosas em bancos de dados relacionais. Ela combina tuplas de duas relações baseando-se em uma condição de junção, tipicamente uma igualdade entre atributos.

Nested Loop Join

Abordagem iterativa simples que compara cada tupla de uma relação com todas as tuplas da outra. Eficiente para relações pequenas ou quando índices estão disponíveis.

Merge Join

Requer que ambas as relações estejam ordenadas pelo atributo de junção. Realiza uma varredura linear sincronizada, sendo muito eficiente para grandes conjuntos ordenados.

Hash Join

Usa funções hash para particionar ambas as relações em buckets correspondentes. Excelente desempenho para grandes volumes quando há memória suficiente.

A escolha do algoritmo de junção adequado pode impactar dramaticamente o desempenho de uma consulta, variando de segundos a horas de execução.

Junção de Loop Aninhado em Bloco

O algoritmo processa a relação externa em **blocos de (M-2) páginas**; para cada bloco externo, varre completamente a relação interna.

Custo

$$b_r + \left\lceil \frac{b_r}{M-2} \right\rceil \times b_s$$

Parâmetros

- **b_r** = blocos da relação externa (r)
- **b_s** = blocos da relação interna (s)
- **M** = buffers disponíveis

ENTRADA: relações **r** (externa) e **s** (interna); **M buffers**

PRÉ-CONDIÇÃO: $M \geq 3$ (M-2 para r, 1 para s, 1 para saída)

PASSO 1 — Iterar sobre blocos da relação externa
para cada chunk de (M - 2) blocos de **r**:
carregar chunk de r na memória

PASSO 2 — Varrer relação interna
para cada bloco **b_s** de **s**:
carregar **b_s** na memória
para cada tupla **t_r** no chunk de **r**:
para cada tupla **t_s** em **b_s**:
se **t_r** e **t_s** satisfaz condição de junção:
adicionar (t_r, t_s) ao buffer de saída
se buffer de saída **cheio**: gravar no disco

SAÍDA: relação resultado da junção **r** e **s**

PÓS-CONDIÇÃO: cada bloco de s é lido $\lceil b_r / (M-2) \rceil$ vezes

Junção de Loop Aninhado em Bloco: Pior Caso

O pior cenário ocorre quando a memória disponível é extremamente limitada, permitindo apenas um bloco de cada relação mais um bloco de saída.

Blocos Mínimos (3)

Apenas 3 blocos de memória disponíveis

Acessos Totais ($b_1 \times b_2$)

Cada bloco externo requer varredura completa da interna

Fórmula de Custo no Pior Caso:

$$Custo = b_r + (b_r \times b_s)$$

Onde **b_r** é o número de blocos da relação externa e **b_s** é o número de blocos da relação interna.

Neste cenário, para cada bloco da relação externa, toda a relação interna precisa ser lida do disco. Isso resulta em um número quadrático de operações de I/O, tornando o algoritmo impraticável para grandes relações.



Este caso ilustra a importância crítica da alocação adequada de memória para operações de junção em SGBDs.

Junção de Loop Aninhado em Bloco: Melhor Caso

O melhor cenário ocorre quando há memória suficiente para carregar completamente a menor relação, eliminando a necessidade de múltiplas varreduras.

Condição Ideal

Memória suficiente para toda a relação externa: $M \geq br + 2$

Varredura Única

Cada relação é lida exatamente uma vez do disco

Performance Máxima

Custo linear minimizado em relação ao tamanho dos dados

Fórmula de Custo no Melhor Caso:

$$Custo = b_r + b_s$$

Este é o cenário ideal onde:

- Toda a relação externa cabe na memória
- Apenas uma varredura da relação interna é necessária
- O número de operações de I/O é minimizado

Implicações práticas:

Otimizadores de consulta sempre tentam alocar a menor relação como externa e maximizar o uso de memória disponível para aproximar-se deste caso ideal.

Junção de Loop Aninhado em Bloco: Caso Geral

No caso geral, a memória disponível permite carregar múltiplos blocos da relação externa, mas não toda a relação. Este é o cenário mais comum na prática.

Fórmula de Custo no Caso Geral:

$$Custo = b_r + \lceil \frac{b_r}{M - 2} \rceil \times b_s$$

Variáveis da Fórmula

- **br:** Número de blocos da relação externa
- **bs:** Número de blocos da relação interna
- **M:** Número de blocos de memória disponíveis
- **M-2:** Blocos utilizáveis para a relação externa (2 reservados para entrada/saída)

Interpretação

O termo $\lceil br/(M-2) \rceil$ representa o número de vezes que precisamos varrer a relação interna. Cada iteração processa (M-2) blocos da relação externa contra toda a relação interna.

A eficiência aumenta significativamente com mais memória disponível. Dobrar a memória pode reduzir o custo pela metade, demonstrando o retorno não-linear do investimento em recursos de memória.

Exemplo de Custos de Junção de Loop Aninhado

Vamos analisar concretamente a operação $\text{depositante} \bowtie \text{cliente}$, com depositante como relação externa.

Dados de Entrada

- Tuplas em cliente: 10.000
- Tuplas em depositante: 5.000
- Blocos de cliente: 400
- Blocos de depositante: 100

Pior Caso

$400 \times 100 + 100$ blocos = 40100 blocos

Caso Geral com M blocos de memória:

$$Custo = 100 + \left\lceil \frac{100}{M - 2} \right\rceil \times 400$$

Análise dos Resultados

Com M=5 blocos: $\lceil 100/3 \rceil \times 400 + 100 = 13.500$ blocos

Com M=12 blocos: $\lceil 100/10 \rceil \times 400 + 100 = 4.100$ blocos

Configuração

Relação Externa: depositante (menor)

Relação Interna: cliente

Melhor Caso

$400 + 100$ blocos = 500 blocos

Impacto da Memória

Aumentar a memória de 5 para 12 blocos reduz o custo em 70%, demonstrando o impacto dramático da alocação de recursos.

Junção de Merge (Merge Join)

A **junção de merge** varre ambas as relações ordenadas simultaneamente, avançando ponteiros e emitindo tuplas que satisfazem a condição de junção.

Custo (relações já ordenadas)

$$Custo = b_r + b_s$$

Parâmetros:

- **b_r** = número de blocos da relação r
- **b_s** = número de blocos da relação s
- Ambas devem estar ordenadas pelo atributo de junção
- Se não ordenadas, aplica-se merge-sort externo primeiro

ENTRADA: relações **r** e **s** ordenadas pelo atributo de junção **A**

PRÉ-CONDIÇÃO: r e s estão ordenadas por A (crescente)

PASSO 1 — Inicialização

p_r ← início de r; p_s ← início de s

PASSO 2 — Varredura sincronizada

enquanto p_r ≠ fim(r) e p_s ≠ fim(s):

se r[p_r].A < s[p_s].A:

avançar p_r

senão se r[p_r].A > s[p_s].A:

avançar p_s

senão: // r[p_r].A = s[p_s].A

PASSO 3 — Produto cartesiano local

marcar posição p_{s_inicio}

enquanto s[p_s].A = r[p_r].A:

emitir (r[p_r], s[p_s])

avançar p_s

avançar p_r

se r[p_r].A = s[p_{s_inicio}].A:

p_s ← p_{s_inicio} // reiniciar para próximo grupo

SAÍDA: relação resultado r ∞ s

PÓS-CONDIÇÃO: cada bloco lido no máximo uma vez

(sem duplicatas no atributo de junção)

Junção Merge: Análise Detalhada

Custo Total

O custo da junção de merge considera duas componentes principais:

$$Custo_{total} = Custo_{mesclagem} + Custo_{ordenação}$$

Custo de Ordenação

Se as relações não estiverem ordenadas:

- Relação r: $b_r \times (2\lceil \log_{M-1}(b_r/M) \rceil + 1)$
- Relação s: $b_s \times (2\lceil \log_{M-1}(b_s/M) \rceil + 1)$

Vantagens

- Performance previsível e linear
- Excelente para grandes volumes ordenados
- Aproveita índices existentes

Custo de Mesclagem

Caso típico: $b_r + b_s$

Cada bloco de ambas as relações é lido exatamente uma vez durante a fase de mesclagem.

Caso com Duplicatas

Se houver muitos valores duplicados no atributo de junção, pode ser necessário fazer backup e reler alguns blocos, aumentando o custo para até $b_r \times b_s$ no pior caso extremo.

Desvantagens

- Requer ordenação prévia (custo adicional)
- Pode ser custoso com muitas duplicatas
- Não ideal para dados não ordenados

Junção de Hash (Hash Join)

Ideia Central

Particionar ambas as relações com a mesma função hash **h**; para cada par de partições correspondentes, construir tabela hash da menor e sondar com a maior.

Custo

$$Custo = 3 \times (b_r + b_s)$$

Parâmetros

- **b_r, b_s** = blocos das relações r e s
- **n** = número de partições
- Requer $n \leq M - 1$
- Menor partição deve caber em memória

ENTRADA: relações **r** e **s**; função hash **h**; **M** buffers

PRÉ-CONDIÇÃO: $n \leq M - 1$; $\lceil b_s/n \rceil + 2 \leq M$

PASSO 1 — Fase de Particionamento (ambas as relações)

para cada tupla **t_r** em **r**:

$i \leftarrow h(t_r.A) \bmod n$

gravar **t_r** na partição **r_i**

para cada tupla **t_s** em **s**:

$i \leftarrow h(t_s.A) \bmod n$

gravar **t_s** na partição **s_i**

PASSO 2 — Fase de Sondagem (por partição)

para $i \leftarrow 0$ até $n - 1$:

carregar partição **s_i** na memória

construir tabela hash **H_i** usando **h'(t.A)**

para cada tupla **t_r** em **r_i**:

buscar **t_r.A** em **H_i**

para cada **t_s** correspondente em **H_i**:

se **t_r** **t_s** satisfaz condição de junção:

emitir (**t_r**, **t_s**)

SAÍDA: relação resultado **r s**

PÓS-CONDIÇÃO: cada bloco lido/gravado **3** vezes no total

Princípio da Junção de Hash

O diagrama ilustra o fluxo completo do algoritmo de hash join, mostrando como as relações são particionadas usando a função hash e posteriormente processadas em pares.

Particionamento Inicial

Relação r dividida em n partições:

$r_0, r_1, \dots, r_{n-1}$

Particionamento Correspondente

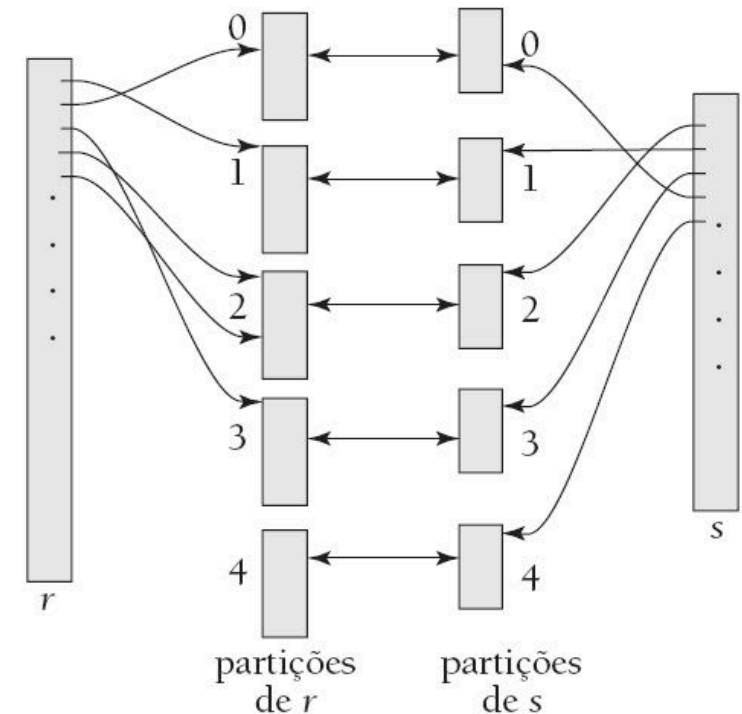
Relação s dividida nas mesmas n partições:

$s_0, s_1, \dots, s_{n-1}$

Processamento por Pares

Junção de r_i com s_i para todo i , combinando apenas partições correspondentes

Garantia de Correção: Tuplas que satisfazem a condição de junção sempre terão o mesmo valor hash, portanto estarão na mesma partição. Isso garante que nenhuma tupla de resultado seja perdida.



- ❏ O número de partições (n) é escolhido de forma que cada partição de construção caiba confortavelmente na memória disponível, tipicamente $n = \lceil b_r / M \rceil$ onde M é o tamanho da memória.

Comparações via Junção de Hash

O número de comparações realizadas durante uma junção de hash é significativamente influenciado pela qualidade da função hash e pela distribuição dos dados.

Caso Ideal

Distribuição uniforme perfeita: Cada partição tem aproximadamente o mesmo tamanho. Número de comparações $\approx (nr \times ns)/n$, onde n é o número de partições.

Caso Típico

Distribuição razoável: Pequenas variações no tamanho das partições. Número de comparações próximo ao ideal com overhead mínimo para tratar desbalanceamento.

Caso Problemático

Distribuição enviesada: Algumas partições muito grandes (skew). Pode degenerar para comparações próximas a $nr \times ns$ se uma partição contém a maioria das tuplas.

Otimizações Práticas

- Uso de funções hash múltiplas
- Re-particionamento de partições grandes
- Amostragem para detectar skew
- Híbridos com nested loop para partições pequenas

Métricas de Qualidade

SGBDs modernos monitoram estatísticas de hash join:

- Fator de desbalanceamento de partições
- Taxa de colisões de hash
- Número de re-particionamentos necessários

Custo da Junção de Hash e Comparação com Merge Join

Vamos consolidar a análise de custos dos principais algoritmos de junção, considerando cenários práticos e trade-offs de desempenho.

Fases do Hash Join

Particionamento, construção e sondagem

Fórmula de Custo do Hash Join:

$$Custo_{hash} = 3(b_r + b_s) + 4n_h$$

Onde n_h é o número de blocos usados para armazenar estruturas auxiliares de hash.

Varreduras por Relação

Cada relação é lida duas vezes

Número de Partições

Tipicamente escolhido como $n \geq br/M$

Hash Join

Melhor para: Grandes volumes de dados não ordenados, junções equi-join, quando há memória suficiente

Custo típico: $3(br + bs)$

Merge Join

Melhor para: Dados já ordenados, índices disponíveis, junções que requerem saída ordenada

Custo típico: $br + bs$ (se já ordenado)

Nested Loop

Melhor para: Relações pequenas, quando índices na relação interna estão disponíveis

Custo típico: $br + (br \times bs)/(M-2)$

❏ A escolha do algoritmo de junção ideal depende de múltiplos fatores: tamanho das relações, disponibilidade de índices, quantidade de memória, existência de ordenação prévia e seletividade da junção.

Avaliação de Expressões em Bancos de Dados

Compreender como o banco de dados processa e executa consultas SQL é fundamental para otimizar o desempenho de sistemas de dados. Uma consulta não é executada de forma instantânea - ela passa por um processo estruturado de avaliação que determina como os dados serão recuperados e processados.

Expressões e Suas Composições

Uma consulta SQL é, essencialmente, um conjunto organizado de **expressões**. Cada expressão representa uma composição de operações individuais que trabalham em conjunto para recuperar e transformar dados.

Quando o banco de dados recebe uma consulta, ele a converte em uma árvore de expressões - uma estrutura hierárquica que representa a ordem lógica das operações. Esta árvore precisa ser avaliada de forma eficiente, e existem estratégias distintas para isso.

Estratégias de Avaliação

O sistema de gerenciamento de banco de dados pode escolher entre duas abordagens principais para processar a árvore de expressões:

Materialização

Gera e armazena **resultados intermediários** em disco. Cada operação é executada completamente antes da próxima, e seus resultados ficam persistidos em relações temporárias.

Pipelining

Passa tuplas adiante para operações ancestrais **durante a execução**, sem esperar a conclusão completa de cada etapa. Funciona como uma linha de produção contínua.

Materialização: Avaliação Passo a Passo

A **avaliação materializada** segue uma abordagem metódica e sequencial. O sistema processa uma operação por vez, sempre começando pelo nível mais baixo da árvore de expressões e subindo gradualmente até completar toda a consulta.

Identificar operação base

Localiza a operação mais básica na árvore de expressões - aquela que não depende de nenhum resultado intermediário

Executar e materializar

Executa completamente a operação e armazena o resultado em uma relação temporária no disco

Subir na hierarquia

Utiliza os resultados materializados como entrada para as operações do nível superior

Repetir até o topo

Continua o processo até que todas as operações sejam executadas e o resultado final seja obtido

Exemplo Prático

Considere uma consulta que precisa:

1. Filtrar contas com saldo < 2500
2. Fazer junção com a tabela cliente
3. Projetar apenas nome_cliente

Na materialização, o sistema:

1. **Calcula e armazena** osaldo<2500(conta)
2. **Lê do disco** e faz junção com cliente
3. **Projeta** o resultado final

Aspectos Críticos da Materialização

✓ Aplicabilidade Universal

A avaliação materializada **sempre funciona**, independentemente da complexidade da consulta ou dos tipos de operações envolvidas. É a abordagem mais robusta e confiável.

Fórmula de Custo Total

$$\text{Custo Total} = \sum_{i=1}^n \text{Custo}_{\text{operação}_i} + \text{Custo}_{\text{I/O intermediário}}$$

Otimização: Buffer Duplo

Uma técnica importante para reduzir o impacto do I/O é o **buffer duplo**. Esta estratégia utiliza dois buffers de saída para cada operação:

- **Buffer A:** Sendo escrito para o disco
- **Buffer B:** Sendo preenchido com novos dados

Isso permite **sobreposição** entre operações de escrita em disco e computação, reduzindo o tempo total de execução da consulta.

❑ Alto Custo de I/O

O maior problema é o **custo elevado de escrita e leitura** de resultados intermediários. Cada operação gera disco I/O adicional, que pode dominar o tempo total de execução.

Pipelining: Processamento Contínuo

A **avaliação em pipeline** representa uma abordagem fundamentalmente diferente. Em vez de processar uma operação completamente antes de iniciar a próxima, o pipelining permite que **múltiplas operações executem simultaneamente**, passando resultados parciais entre si de forma contínua.

Filtrar

Processa tuplas e passa resultados imediatamente

Junção

Recebe tuplas filtradas e gera junções incrementalmente

Projetar

Aplica projeção conforme tuplas chegam

Vantagens e Limitações

Benefícios do Pipelining

- **Custo computacional reduzido** - não há necessidade de armazenar relações temporárias
- **Menor latência** - resultados começam a aparecer mais cedo
- **Uso eficiente de memória** - apenas dados ativos ocupam RAM

Quando Não é Possível

- **Operações de ordenação** - precisam ver todos os dados antes de gerar saída
- **Junção hash** - requer construção completa da tabela hash
- **Agregações complexas** - podem precisar de dados completos

📌 **Requisito chave:** Para que o pipelining seja eficiente, os algoritmos de avaliação devem ser capazes de gerar tuplas de saída à medida que recebem tuplas de entrada, sem necessidade de acumular todos os dados primeiro.

Pipelining Controlado por Demanda

Ideia Central

Cada operador implementa a **interface iterador** (`open` / `next` / `close`). A operação raiz **puxa tuplas sob demanda**, propagando chamadas recursivamente para baixo na árvore de expressões.

Vantagem principal: elimina a materialização de relações intermediárias, reduzindo drasticamente o I/O.

Os Três Métodos

`open()`

Inicializa a operação e prepara estruturas

`next()`

Retorna a próxima tupla ou $\emptyset$ quando esgotado

`close()`

Libera recursos e finaliza a execução

ENTRADA: árvore de operadores O com raiz R ; relações **base** nas folhas

PRÉ-CONDIÇÃO: cada operador implementa `{open, next, close}`

PASSO 1 — Inicialização (top-down)

`R.open()`

// propaga recursivamente para filhos:

// `filho_esq.open()`; `filho_dir.open()`; ...

PASSO 2 — Produção de **tuplas** (pull loop)

enquanto verdadeiro:

`t ← R.next()`

se `t =  $\emptyset$` : interromper // pipeline esgotado

entregar `t` ao cliente

// Internamente, `next()` de cada operador:

// chama `next()` nos filhos conforme necessário,

// processa as tuplas recebidas,

// retorna uma tupla de saída ou $\emptyset$

PASSO 3 — Finalização (top-down)

`R.close()`

// propaga recursivamente para filhos

SAÍDA: sequência de tuplas resultado da consulta

PÓS-CONDIÇÃO: nenhuma relação intermediária gravada em **disco**
(exceto operadores bloqueantes: `sort`, `hash` **join**)

Pipelining Controlado por Produtor

O modelo **controlado por produtor** (push-based) inverte a direção de controle. Aqui, os operadores **produzem tuplas ativamente** e as empurram para seus pais assim que ficam disponíveis, sem esperar por solicitações.

Produção Ativa

Operadores filhos geram tuplas rapidamente e as colocam em buffers compartilhados

Controle de Fluxo

Se buffer enche, produtor aguarda; se esvazia, consumidor aguarda

Comparação: Demanda vs. Produtor

Controlado por Demanda

- Controle top-down (puxar)
- Processa sob solicitação
- Mais fácil de implementar
- Menor overhead de sincronização

Desafios do Modelo Produtor

O sistema precisa implementar um **escalonador sofisticado** que monitore o estado dos buffers e decida quais operações devem executar a cada momento. Operações que têm espaço em seus buffers de saída e podem processar mais tuplas de entrada recebem prioridade. Este escalonamento dinâmico garante que o pipeline continue fluindo sem bloqueios desnecessários.

Buffer Intermediário

Mantido entre operadores - filho escreve, pai consome conforme capacidade

Escalonamento

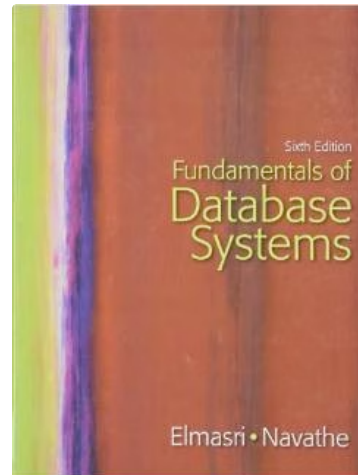
Sistema coordena operações com espaço disponível para processar

Controlado por Produtor

- Controle bottom-up (empurrar)
- Produção contínua e agressiva
- Melhor paralelismo potencial
- Requer gestão cuidadosa de buffers

📌 **Consideração importante:** Ambos os modelos de pipelining requerem algoritmos de avaliação especialmente projetados que possam operar incrementalmente, gerando resultados parciais conforme processam os dados de entrada.

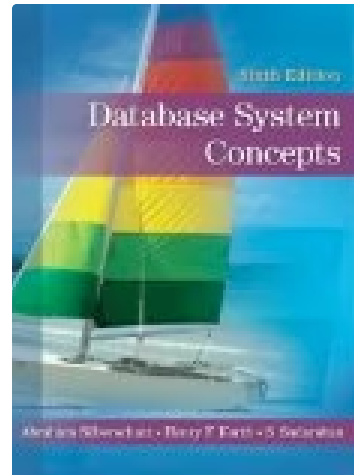
Referências



Elmasri & Navathe

Fundamentals of Database Systems

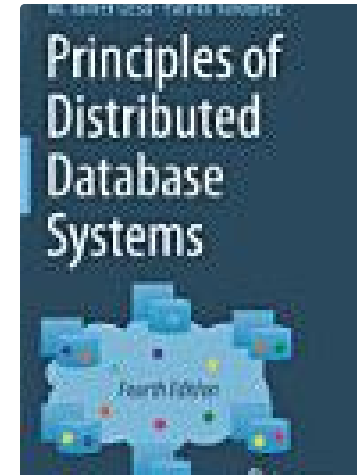
Pearson, 2016



Korth, Sudarshan & Silberschatz

Database System Concepts

McGraw-Hill, 2019



Örsal & Valduriel

Principles of Distributed Database Systems

Springer Nature, 2019